CORE – 14: Algorithm Design Techniques (Unit – 3)

Greedy Technique: Greedy Algorithm solves problems by making the best choice that seems best at the particular moment without worrying about the effect of the current decision in future. This approach never reconsiders the choices taken previously.

Applications of Greedy Algorithms

1. Finding an optimal solution (Activity selection, Fractional Knapsack, Job Sequencing, Huffman Coding).
2. Finding close to the optimal solution for NP-Hard problems like TSP.

Fractional Knapsack Problem: Given weights and values of n items, we need to put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

Steps to solve the Fractional Problem:

1. Compute the value per pound $\mathbf{v_i / w_i}$ for each item. (v_i is value of item i and w_i weight of item i)
2. Take as much as possible of the item with the highest value per pound.
3. If the supply of that element is exhausted, we take as much as possible of the element with the next value per pound.
4. Let capacity of the Knapsack is **W** i.e. maximum of W weight can be taken in the knapsack.

Example: Consider 5 items along their respective weights and values:
I = (I1, I2, I3, I4, I5), w = (5, 10, 20, 30, 40), v = (30, 20, 100, 90,160)
The capacity of knapsack **W = 60**
Find the maximum value which can be carried by the knapsack?

Solution: arrange the value of p_i in decreasing order.

Item	w_i	v_i	$P_i = v_i/w_i$
I_1	5	30	6.0
I_3	20	100	5.0
I_5	40	160	4.0
I_4	30	90	3.0
I_2	10	20	2.0

Fill the knapsack according to the decreasing value of p_i. First, we choose the item $\mathbf{I_1}$ whose weight is 5. Then choose item $\mathbf{I_3}$ whose weight is 20. Now, the total weight of knapsack is 20 + 5 = 25
Now the next item is $\mathbf{I_5}$, and its weight is 40, but we want only 35, so we chose the fractional part of it,
Hence, maximum value = 30 + 100 + 35x4 = **270**

Algorithm - Fractional Knapsack (Array v, Array w, int W)

1. for i = 1 to size (v)
   do p [i] = v[i] / w[i]
2. Sort-Descending (p)
3. i ← 1
4. while (W > 0)
   a) **do amount = min (W, w[i])**
   b) solution[i] = amount
   c) W = W - amount
   d) i ← i + 1
5. return solution

Job Sequencing with Deadlines: Given an array of jobs where every job has a deadline and associated profit if the job is finished before the deadline. It is also given that every job takes the single unit of time, so the minimum possible deadline for any job is 1. Job Sequencing deals with the problem that how to maximize total profit if only one job can be scheduled at a time.

Example: Consider a set of given jobs as shown in the. Find a sequence of jobs, which will be completed within their deadlines and will give maximum profit. Each job is associated with a deadline and profit.

Job	Deadline	Profit
J_1	2	60
J_2	1	100
J_3	3	20
J_4	2	40
J_5	1	20

Solution: To solve this problem, the given jobs are sorted according to their profit in a descending order. After sorting, the jobs are ordered as shown in following table.

Job	Deadline	Profit
J_2	1	100
J_1	2	60
J_4	2	40
J_3	3	20
J_5	1	20

- From this set of jobs, first we select ***J_2***, as it can be completed within its deadline and contributes maximum profit.
- Next, ***J_1*** is selected as it gives more profit compared to ***J_4***.
- In the next clock, ***J_4*** cannot be selected as its deadline is over, hence ***J_3*** is selected as it executes within its deadline.
- The job ***J_5*** is discarded as it cannot be executed within its deadline.

Thus, the solution is the sequence of jobs (***J_2, J_1, J_3***), which can be executed within their deadline and gives maximum profit.
Total profit of this sequence is **100 + 60 + 20 = 180**.

Algorithm

1) Sort all jobs in decreasing order of profit.
2) Iterate on jobs in decreasing order of profit. For each job, do the following:
   a) Find a time slot i, such that slot is empty and i < deadline and i is greatest. Put the job in this slot and mark this slot filled.
   b) If no such i exists, then ignore the job.

Huffman Codes: Huffman coding is a **variable-length code**, which is mainly used for compression. The code length is related to the frequency of characters in a massage. **Most frequent characters have the smallest codes and less frequent characters have the longer codes**.

There are mainly two parts in Huffman Coding;

1. Build a Huffman tree from the input characters.
2. Traverse the tree to find codes.

For example, consider some strings “YYYZXXYYX”, the frequency of character Y is larger than X and the character Z has the least frequency. So the length of the code for Y is smaller than X, and code for X will be smaller than Z.

Complexity for assigning the code for each character according to their frequency is **O(n log n)**

Algorithm for creating the Huffman Tree-

Step 1- Create a leaf node for each character and build a min heap using all the nodes (The frequency value is used to compare two nodes in min heap)

Step 2- Repeat Steps 3 to 5 while heap has more than one node

Step 3- Extract two nodes, say x and y, with minimum frequency from the heap

Step 4- Create a new internal node z with x as its left child and y as its right child. Also frequency(z) = frequency(x) + frequency(y)

Step 5- Add z to min heap

Step 6- Last node in the heap is the root of Huffman tree

Example: Create Huffman Tree for the following characters along with their frequencies and find the Huffman code for each character.

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1

Solution: Step 1- Create a leaf node for each character and build a min heap using all the nodes (The frequency value is used to compare two nodes in min heap)

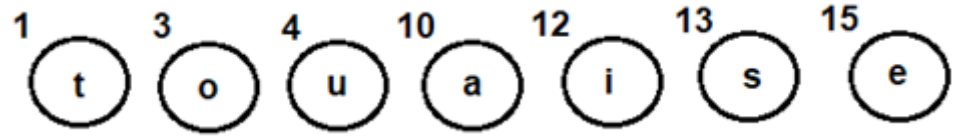


Fig 1: Leaf nodes for each character

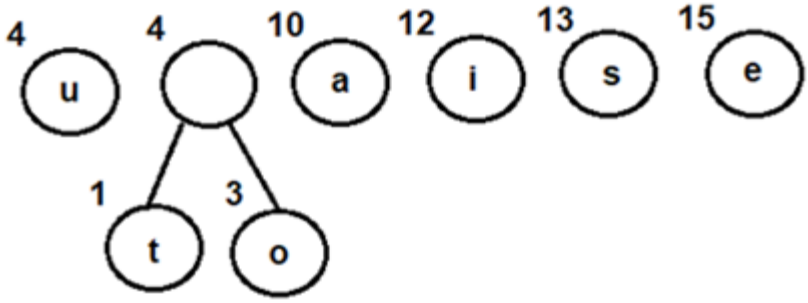


Fig 2: Combining nodes o and t

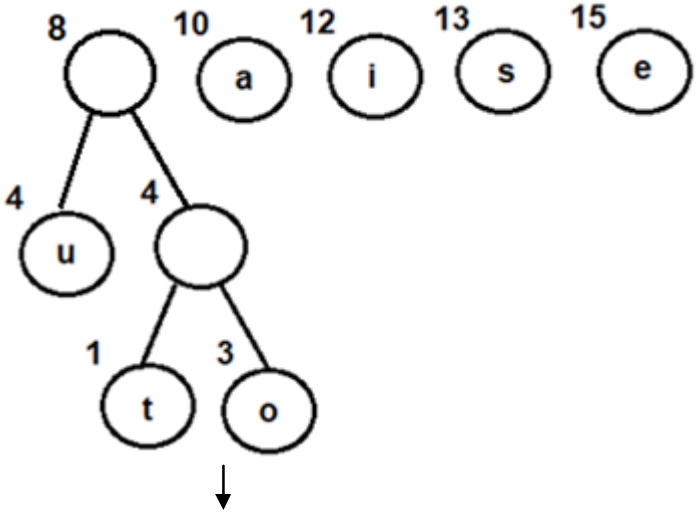


Fig 3: Combining node u with an internal node having 4 as frequency

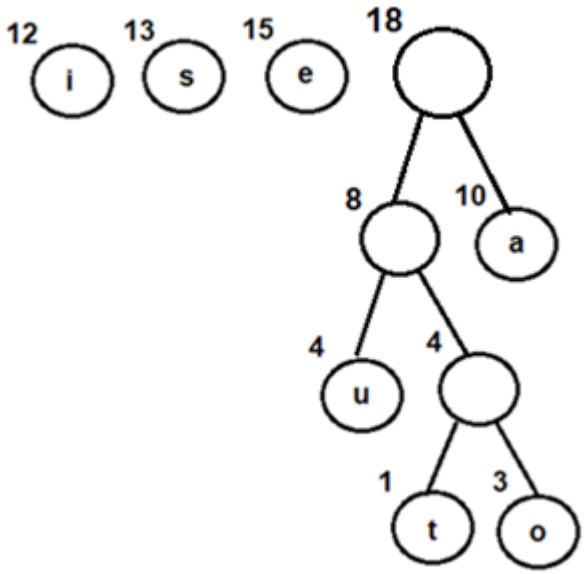


Fig 4: Combining node a with an internal node having 8 as frequency

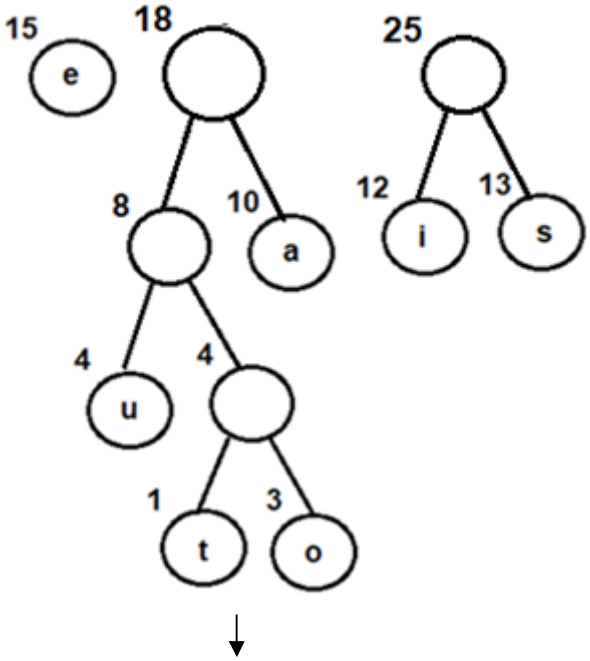


Fig 5: Combining nodes i and s

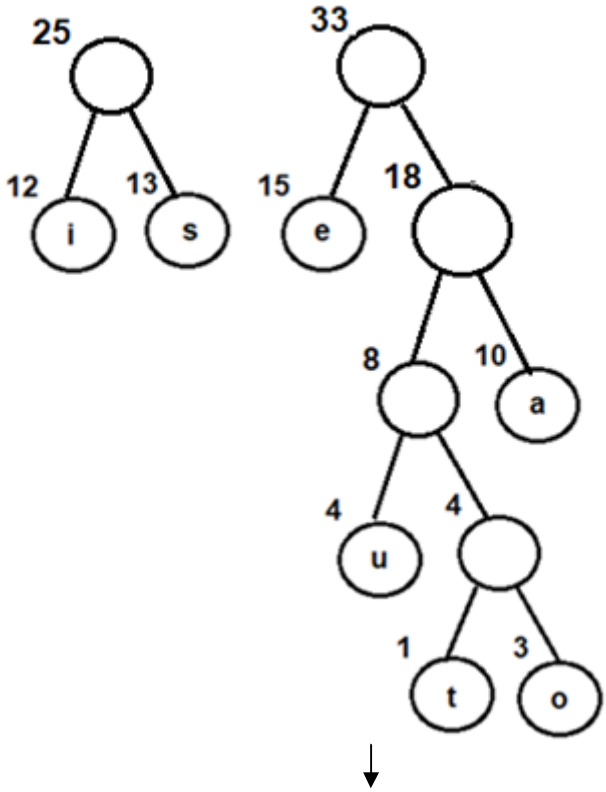


Fig 6: Combining node e with an internal node having 18 as frequency

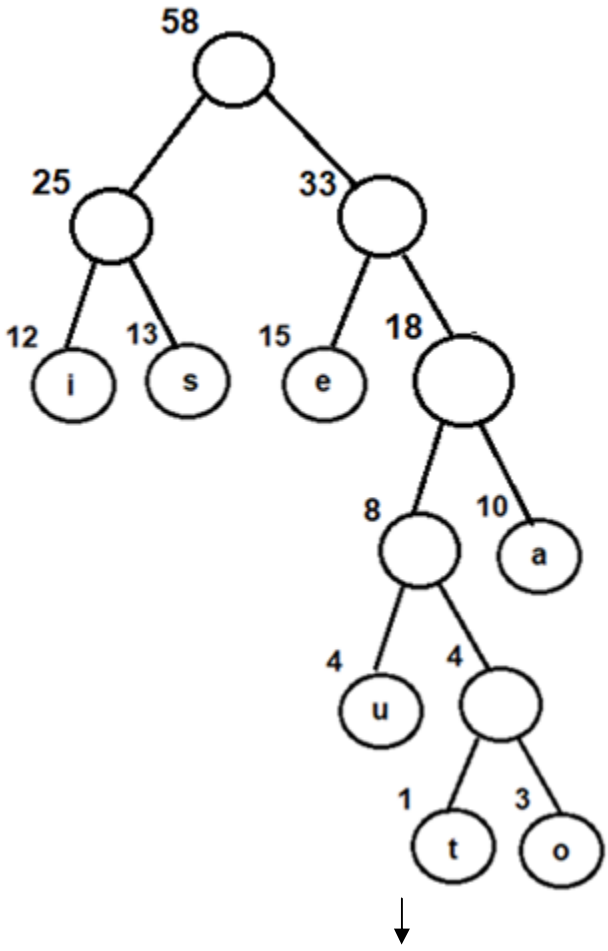
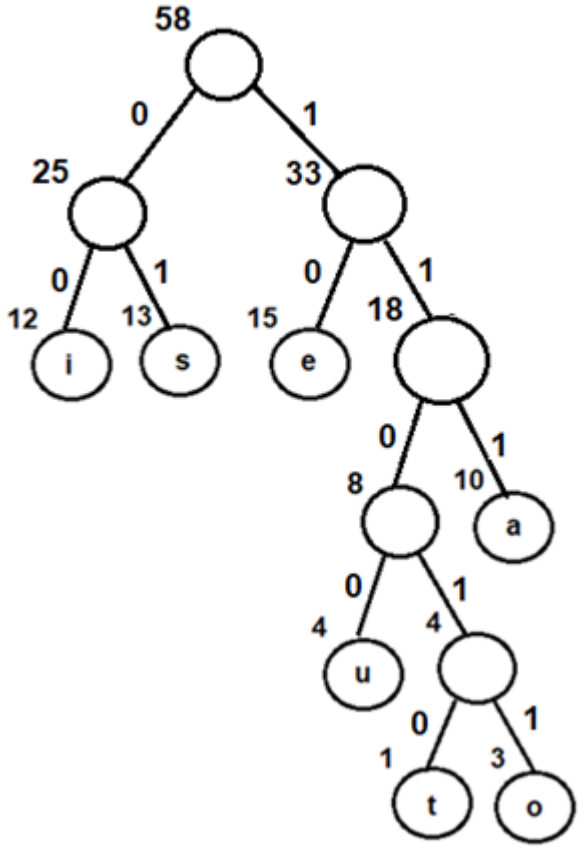


Fig 7: Final Huffman tree obtained by combining internal nodes having 25 and 33 as frequency



Assigning 0 to the left child and 1 to the right child and finding the Huffman codes for the characters.

Characters	Binary Codes
i	00
s	01
e	10
u	1100
t	11010
o	11011
a	111

Dynamic Programming: Dynamic Programming is the most powerful design technique for solving optimization problems. **Divide & Conquer** algorithm partition the problem into **disjoint subproblems**, solve the subproblems recursively and then combine their solution to solve the original problems. Dynamic Programming is used when the **subproblems are not independent**, e.g. when they share the same subproblems. In this case, divide and conquer may do more work than necessary, because it solves the same sub problem multiple times. Dynamic Programming solves each subproblems just once and stores the result in a table so that it can be repeatedly retrieved if needed again. It is a **Bottom-up approach.** We solve all possible small problems and then combine to obtain solutions for bigger problems.

Applications of dynamic programming

1. Matrix Chain Multiplication
2. Longest common subsequence.

Matrix Chain Multiplication: Given a sequence of matrices, find the most efficient way to multiply these matrices together. The problem is not actually to perform the multiplications, but merely to decide in which order to perform the multiplications.

We have many options to multiply a chain of matrices because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result will be the same. For example, if we had four matrices A, B, C, and D, we would have:

(ABC)D = (AB)(CD) = A(BCD) = ...

However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency.

For example, suppose A is a 10 × 30 matrix, B is a 30 × 5 matrix, and C is a 5 × 60 matrix. Then,

(AB)C = (10×30×5) + (10×5×60) = 1500 + 3000 = 4500 operations

A(BC) = (30×5×60) + (10×30×60) = 9000 + 18000 = 27000 operations.

Clearly the first parenthesization requires less number of operations.

Given an array **p[ ]** which represents the chain of matrices(i.e. it contains the order of all matrices) such that the **i^{th} matrix A_i** is of dimension **p[i-1] x p[i].** We need to write a function **MatrixChainOrder()** that should return the minimum number of multiplications needed to multiply the chain.

Step1 - Structure of an optimal parenthesization:

Let $A_{i....j}$ be the product of A_i A_{i+1}....A_j where i≤ j and **A_i is a matrix of order p[i-1] x p[i],** similarly A_{i+1} is a matrix of order p[i] x p[i+1] and so on

If i < j then any parenthesization of the product A_i A_{i+1} .....A_j must split that the product between A_k and A_{k+1} for some integer k in the range i ≤ k ≤ j. That is for some value of k, we first compute the matrices $A_{i.....k}$ & $A_{k+1....j}$ and then multiply them together to produce the final product $A_{i....j}$.

Step 2: A Recursive Solution:

Let m [i, j] = minimum number of scalar multiplication needed to compute the matrix $A_{i....j}$.

If i = j the chain consist of just one matrix $A_{i....i}$ = A_i so no scalar multiplication are necessary to compute the product. Thus m [i, j] = 0 for i= 1, 2, 3....n.

If i<j we assume that to optimally parenthesize the product we split it between A_k and A_{k+1} where i≤ k ≤j. Then m [i,j] equals the minimum cost for computing the subproducts $A_{i....k}$ and $A_{k+1....j}$+ cost of multiplying them together. We know A_i has dimension p_{i-1} x p_i, so computing the product $A_{i....k}$ and $A_{k+1....j}$takes p_{i-1} p_k p_j scalar multiplication, we obtain

$$\mathbf{m\,[i, j] = m\,[i, k] + m\,[k + 1, j] + p_{i-1}\ p_k\ p_j}$$

There are only (j-i) possible values for 'k' namely k = i, i+1.....j-1. Since the optimal parenthesization must use one of these values for 'k' we need only check them all to find the best.

So the minimum cost of parenthesizing the product A_i A_{i+1}.....A_j becomes

$$m[i,j] = \begin{cases} 0 & \text{if } i = j\ , \\ \min_{i \le k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\} & \text{if } i < j\ . \end{cases}$$

To construct an optimal solution, let us define s [i,j] to be the value of 'k' at which we can split the product A_i A_{i+1} .....A_j to obtain an optimal parenthesization i.e.

s [i, j] = k such that **m [i,j] = m [i, k] + m [k + 1, j] + p_{i-1} p_k p_j**

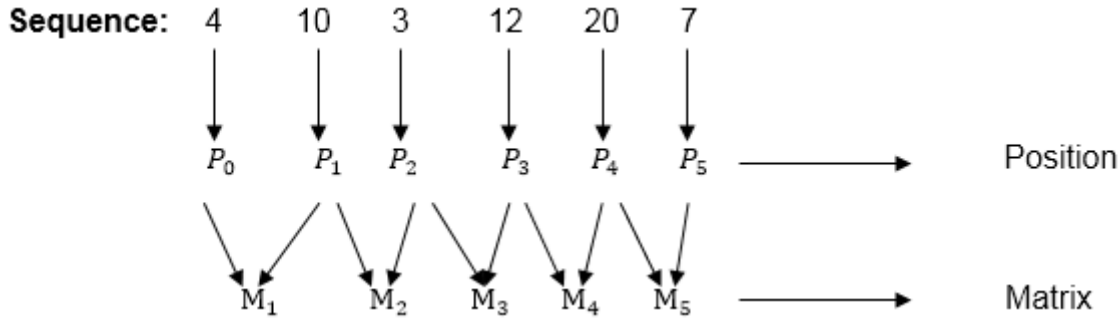
Example: We are given the sequence {4, 10, 3, 12, 20, and 7}. The matrices have size 4 x 10, 10 x 3, 3 x 12, 12 x 20, 20 x 7. We need to compute m [i,j] where 1≤ i ≤ 5 and 1≤ j ≤ 5. We know m [i, i] = 0 for all i.

Solution: Here, p[ ] = {{4, 10, 3, 12, 20, 7}
i.e. p[0]=4, p[1]=10, p[2]=3, p[3]=12, p[4]=20, p[5]=7

Let us proceed with working away from the diagonal. We compute the optimal solution for the product of 2 matrices.

1	2	3	4	5	
0					1
	0				2
		0			3
			0		4
				0	5

(Matrix m[i, j])



Here P_0 to P_5 are Position and A_1 to A_5 are matrices. (A_i is a matrix of size $p_{i-1}xp_i$)

Calculation of Product of 2 matrices:

1. m (1,2) = m_1xm_2 = 4x10x3= 120
2. m(2, 3)= m_2xm_3 =10x3x12= 360
3. m(3, 4)= m_3xm_4=3 x12x20= 720
4. m(4,5)=m_4xm_5 =12x20x7= 1680

1	2	3	4	5	
0	120				1
	0	360			2
		0	720		3
			0	1680	4
				0	5

Matrix m[i, j]

2	3	4	5	
1				1
	2			2
		3		3
			4	4

Matrix s[i, j]

We initialize the diagonal element with equal i,j value with '0'.
After that second diagonal is sorted out and we get all the values corresponded to it
Now the third diagonal will be solved out in the same way.

Calculation of Product of 3 matrices:

$$M[1,3] = \min \begin{cases} M[1,2] + M[3,3] + p_0 p_2 p_3 = 120 + 0 + 4.3.12 = 264 \\ M[1,1] + M[2,3] + p_0 p_1 p_3 = 0 + 360 + 4.10.12 = 840 \end{cases}$$

So, m[1,3]= 264 and s[1,3] = 2

$$M[2,4] = \min \begin{cases} M[2,3] + M[4,4] + p_1 p_3 p_4 = 360 + 0 + 10.12.20 = 2760 \\ M[2,2] + M[3,4] + p_1 p_2 p_4 = 0 + 720 + 10.3.20 = 1320 \end{cases}$$

So, m[2,4] = 1320 and s[2,4] = 2

$$M[3,5] = \min \begin{cases} M[3,4] + M[5,5] + p_2 p_4 p_5 = 720 + 0 + 3.20.7 = 1140 \\ M[3,3] + M[4,5] + p_2 p_3 p_5 = 0 + 1680 + 3.12.7 = 1932 \end{cases}$$

So, m[3,5] = 1140 and s[3,5] = 4

1	2	3	4	5	
0	120	264			1
	0	360	1320		2
		0	720	1140	3
			0	1680	4
				0	5

Matrix m[i, j]

2	3	4	5	
1	2			1
	2	2		2
		3	4	3
			4	4

Matrix s[i, j]

Now Product of 4 matrices:

$$M[1,4] = \min \begin{cases} M[1,3] + M[4,4] + p_0p_3p_4 = 264 + 0 + 4.12.20 = 1224 \\ M[1,2] + M[3,4] + p_0p_2p_4 = 120 + 720 + 4.3.20 = 1080 \\ M[1,1] + M[2,4] + p_0p_1p_4 = 0 + 1320 + 4.10.20 = 2120 \end{cases}$$

So, m[1,4] = 1080 and s[1,4] = 2

$$M[2,5] = \min \begin{cases} M[2,4] + M[5,5] + p_1p_4p_5 = 1320 + 0 + 10.20.7 = 2720 \\ M[2,3] + M[4,5] + p_1p_3p_5 = 360 + 1680 + 10.12.7 = 2880 \\ M[2,2] + M[3,5] + p_1p_2p_5 = 0 + 1140 + 10.3.7 = 1350 \end{cases}$$

So, m[2,5] = 1350 and s[2,5] = 2

1	2	3	4	5	
0	120	264	1080		1
	0	360	1320	1350	2
		0	720	1140	3
			0	1680	4
				0	5

Matrix m[i, j]

2	3	4	5	
1	2	2		1
	2	2	2	2
		3	4	3
			4	4

Matrix s[i, j]

Now Product of 5 matrices:

$$M[1,5] = \min \begin{cases} M[1,4] + M[5,5] + p_0p_4p_5 = 1080 + 0 + 4.20.7 = 1640 \\ M[1,3] + M[4,5] + p_0p_3p_5 = 264 + 1680 + 4.12.7 = 2280 \\ M[1,2] + M[3,5] + p_0p_2p_5 = 120 + 1140 + 4.3.7 = 1344 \\ M[1,1] + M[2,5] + p_0p_1p_5 = 0 + 1350 + 4.10.7 = 1630 \end{cases}$$

So, m[1,5] = 1344 and s[1,5] = 2

1	2	3	4	5	
0	120	264	1080	**1344**	1
	0	360	1320	1350	2
		0	720	1140	3
			0	1680	4
				0	5

Final matrix m[i, j]

2	3	4	5	
1	2	2	2	1
	2	2	2	2
		3	4	3
			4	4

Final matrix s[i, j]

Algorithm of Matrix Chain Multiplication - MatrixChainOrder (p)

1. n ← length[p]-1 //n = number of matrices
2. for i ← 1 to n
   do m [i, i] ← 0
3. for l ← 2 to n // l is the chain length
   do for i ← 1 to n-l + 1
   do j ← i+ l -1
   m[i,j] ← ∞
   for k ← i to j-1
   do q ← m [i, k] + m [k + 1, j] + $p_{i-1}\ p_k\ p_j$
   If q < m [i,j]
   then m [i,j] ← q
   s [i,j] ← k
4. return m and s.

Longest Common Sequence (LCS): A subsequence of a given sequence is just the given sequence with some elements left out. Given two sequences X and Y, we say that the sequence Z is a common sequence of X and Y if Z is a subsequence of both X and Y. In the longest common subsequence problem, we are given two sequences $X = (x_1\ x_2....x_m)$ and $Y = (y_1\ y_2\ y_n)$ and wish to find a maximum length common subsequence of X and Y.

Given a sequence $X = (x_1\ x_2.....x_m)$, we define the ith prefix of X for i = 1, 2...m as $X_i = (x_1\ x_2.....x_i)$. e.g. if X = (A, B, C, B, C, A, B, C) then X_4 = (A, B, C, B).

Optimal Substructure of an LCS: Let $X = (x_1\ x_2....x_m)$ and $Y = (y_1\ y_2.....y_n)$ be the sequences and let $Z = (z_1\ z_2.....z_k)$ be any LCS of X and Y.

- If $x_m = y_n$, then $z_k=x_m=y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}
- If $x_m \neq y_n$, $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y.
- If $x_m \neq y_n$, $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1}

Computing the length of an LCS: let two sequences $X = (x_1\ x_2.....x_m)$ and $Y = (y_1\ y_2..... y_n)$ as inputs. It stores the c[i,j] values in the table c [0.....m,0.....n]. Table b[1.....m, 1.....n] is maintained which help us to construct an optimal solution. c[m, n] contains the length of an LCS of X,Y.

Algorithm: LCS-LENGTH (X, Y)

1. m ← length [X]
2. n ← length [Y]
3. for i ← 1 to m
   do c [i,0] ← 0
4. for j ← 0 to n
   do c [0,j] ← 0
5. for i ← 1 to m
   do for j ← 1 to n
   do if $x_i = y_j$ then
   c [i,j] ← c [i-1,j-1] + 1
   b [i,j] ← "↖"
   else if c[i-1,j] ≥ c[i,j-1] then
   c [i,j] ← c [i-1,j]
   b [i,j] ← "↑"
   else c [i,j] ← c [i,j-1]
   b [i,j] ← "← "
6. return c and b.

The time complexity of LCS-LENGTH is O(mn).

Example: X = "ABCBDAB"
Y = "BDCABA"
Find the LCS.

i	j	0	1 (B)	2 (D)	3 (C)	4 (A)	5 (B)	6 (A)
0	X_i	0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

PRINT-LCS (b, X, i, j)

1. if i = 0 or j = 0 then return
2. if b [i,j] = ' ↖ ' then PRINT-LCS (b, X, i-1, j-1)
   print x_i
   else if b [i,j] = ' ↑ ' then PRINT-LCS (b,X,i-1,j)
   else PRINT-LCS (b, X, i, j-1)

The time complexity of PRINT-LCS is O(m+n).